Lab_sheet-04

Anish Tiwari

Section→C

Roll no.→11

Experiment No. 4

Experiment Name

Frequency Domain Image Filtering using Fourier Transform

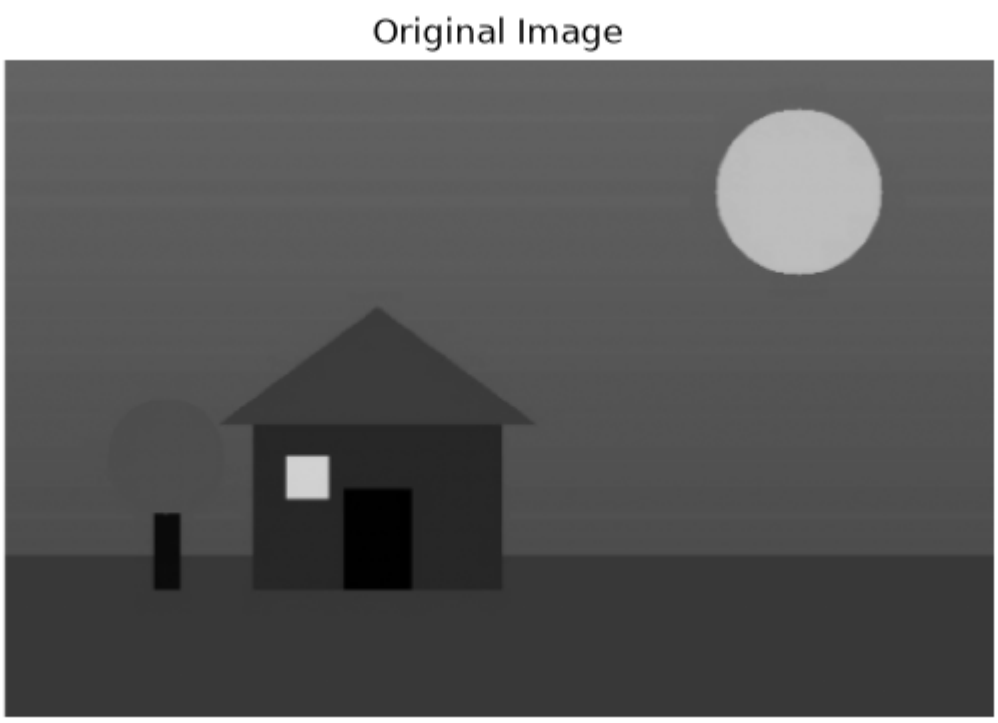
Program Code and Output

Step 1: Libraries import + grayscale image load

In [9]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load grayscale image
image = cv2.imread("my_pic01.png", cv2.IMREAD_GRAYSCALE)

# Display original image
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.show()
```



Step 2: Compute DFT

In [10]:
```
# Compute Discrete Fourier Transform
dft = np.fft.fft2(image)

print("DFT computed successfully")
```

```
DFT computed successfully
```

Step 3: Shift zero-frequency component to center

In [11]:
```
# Shift zero-frequency component to the center
dft_shift = np.fft.fftshift(dft)

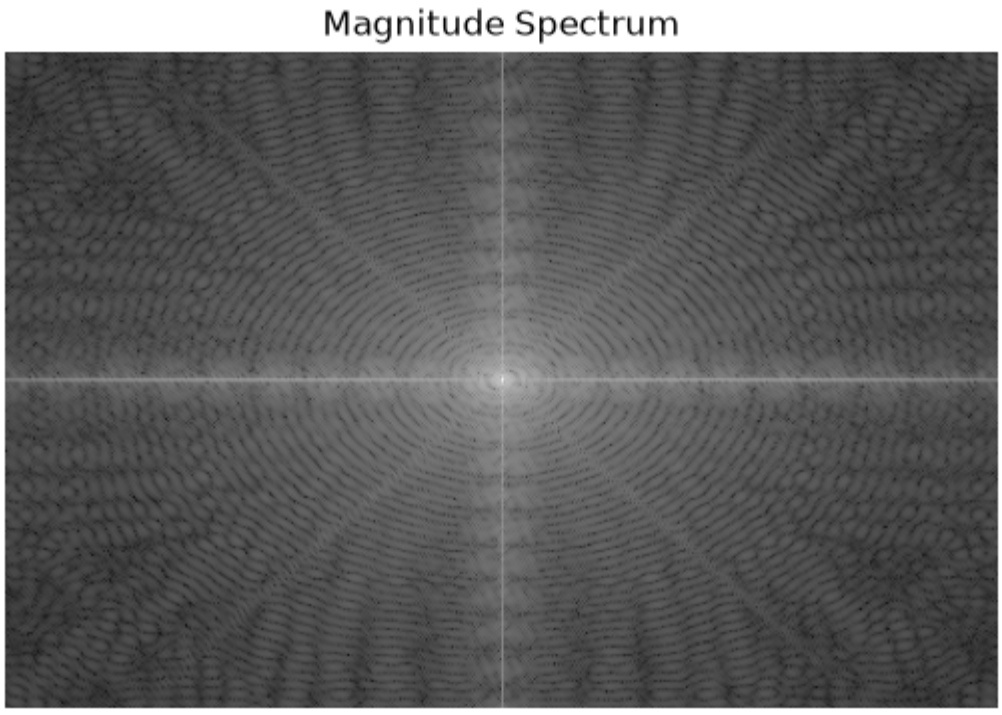
print("Frequency spectrum shifted successfully")
```

```
Frequency spectrum shifted successfully
```

Step 4: Display Magnitude Spectrum

In [12]:
```
# Calculate magnitude spectrum
magnitude_spectrum = 20 * np.log(np.abs(dft_shift) + 1)

# Display spectrum
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.show()
```



Step 5: Low-Pass Frequency Filter

In [13]:
```
rows, cols = image.shape
crow, ccol = rows // 2, cols // 2

# Create mask
mask_low = np.zeros((rows, cols), np.uint8)

# Keep low frequencies around center
radius = 50
cv2.circle(mask_low, (ccol, crow), radius, 1, -1)

# Apply Low-Pass Filter
low_pass = dft_shift * mask_low
```

Step 6: High-Pass Frequency Filter

In [14]:
```
# Create High-Pass mask
mask_high = np.ones((rows, cols), np.uint8)

# Remove low frequencies from center
radius = 50
cv2.circle(mask_high, (ccol, crow), radius, 0, -1)

# Apply High-Pass Filter
high_pass = dft_shift * mask_high
```

Step 7: Inverse Fourier Transform

In [15]:
```
# Low-pass inverse transform
low_pass_shift = np.fft.ifftshift(low_pass)
low_pass_image = np.fft.ifft2(low_pass_shift)
low_pass_image = np.abs(low_pass_image)

# High-pass inverse transform
high_pass_shift = np.fft.ifftshift(high_pass)
high_pass_image = np.fft.ifft2(high_pass_shift)
high_pass_image = np.abs(high_pass_image)
```

Step 8: Compare all images

In [16]:
```
plt.figure(figsize=(12, 8))

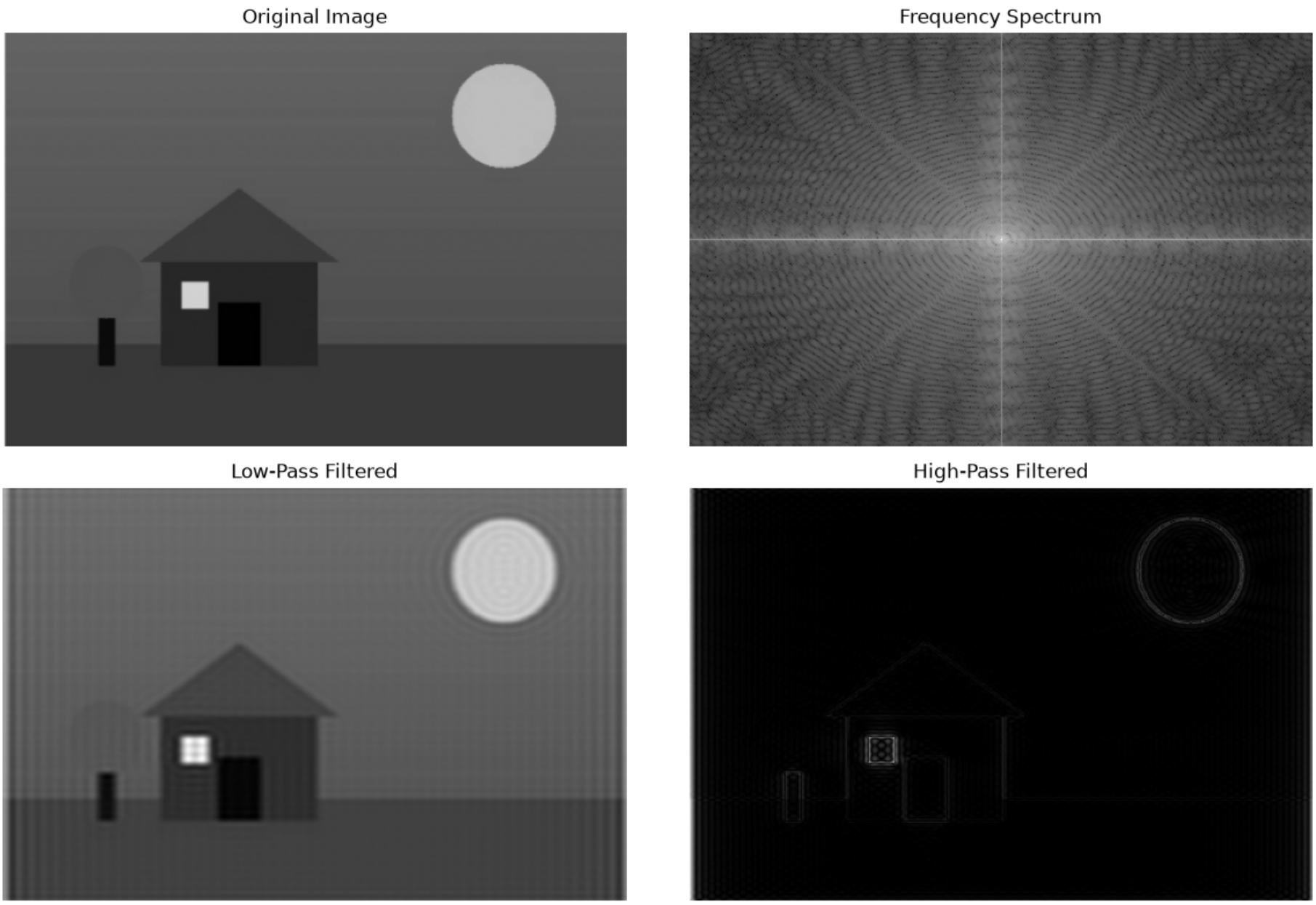
plt.subplot(2, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Frequency Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(low_pass_image, cmap="gray")
plt.title("Low-Pass Filtered")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(high_pass_image, cmap="gray")
plt.title("High-Pass Filtered")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



Step 9: Observation:

In [17]:
```
print("""1. Original image contains both low-frequency and high-frequency components.

2. Low-pass filtering removes high-frequency components, resulting
   in a smoother image and reducing noise.

3. High-pass filtering removes low-frequency components and highlights
   edges and fine details.

4. The frequency spectrum shows the distribution of frequency
   components in the image. """)
```

```
1. Original image contains both low-frequency and high-frequency components.

2. Low-pass filtering removes high-frequency components, resulting
   in a smoother image and reducing noise.

3. High-pass filtering removes low-frequency components and highlights
   edges and fine details.

4. The frequency spectrum shows the distribution of frequency
   components in the image.
```

Step 10: Interpretation

Frequency domain filtering provides an effective method for image enhancement and noise removal. Fourier Transform converts the image from spatial domain to frequency domain, where specific frequency components can be selectively removed or preserved. Low-pass filtering is useful for smoothing and noise reduction, while high-pass filtering is useful for edge enhancement and feature extraction.